Throws

`std::system_error` if the effects can't be achieved. If the `lock` object is unlocked during the call to `wait_until()`, it's locked again on exit, even if the function exits via an exception.

Note

The spurious wakeups mean that a thread calling `wait_until()` may wake even though no thread has called `notify_one()` or `notify_all()`. It's therefore recommended that the overload of `wait_until()` that takes a predicate is used in preference where possible. Otherwise, it's recommended that `wait_until()` be called in a loop that tests the predicate associated with the condition variable. There's no guarantee as to how long the calling thread will be blocked, only that if the function returns `false`, then `Clock::now()` returns a time equal to or later than `absolute_time` at the point at which the thread became unblocked.

Synchronization

Calls to `notify_one(), notify_all(), wait(), wait_for()`, and `wait_until()` on a single `std::condition_variable_any` instance are serialized. A call to `notify_one()` or `notify_all()` will only wake threads that started waiting *prior* to that call.

Std::Condition_Variable_Any::Wait_Until Member Function Overload - That Takes a Predicate

Waits until the `std::condition_variable_any` is woken by a call to `notify_one()` or `notify_all()` and the predicate is `true`, or until the specified time has been reached.